

ASEN 5264 – DMU

Homework #3

Bijan Jourabchi

Problem 1:

- a) Given U^* we can extract the optimal policy π^* from U^* with the following expression:

$$\pi^*(s) = \arg \max_{a \in A} (R(s, a) + \gamma \mathbb{E}[U^*(s)])$$

- b) We are given an MDP defined by,

$$S = \mathbb{R}, \quad A = \{0, 0.1, 1\}, \quad \gamma = 1$$

$$T(s' | s, a) = \mathcal{U}([s + a, s + a + 1])$$

$$R(s, a) = -\mathbb{1}(s \leq 1) - a$$

$$U^{\tilde{\pi}}(s) = -2 \cdot \mathbb{1}(s \leq 1)$$

Where $\mathcal{U}([s + a, s + a + 1])$ represents a uniform distribution from $s + a$ to $s + a + 1$, and $\mathbb{1}(s \leq 1) = 1$ if $s \leq 1$ and 0 otherwise. Given $s = 0.8$, we must find the action which maximizes $Q^{\tilde{\pi}}(s, a)$, which is given by

$$Q^{\tilde{\pi}}(s, a) = R(s, a) + \sum_{s'} T(s' | s, a) U^{\tilde{\pi}}(s')$$

Since there are only three actions, we can compute $Q^{\tilde{\pi}}(s = 0.8, a)$ for $\forall a \in A$. Since, S and T are continuous, we also note that the summation will be an integral over all of the possible future state s' . Thus, our expression for $Q^{\tilde{\pi}}(s, a)$ becomes

$$Q^{\tilde{\pi}}(0.8, a) = R(0.8, a) + \int_{-\infty}^{\infty} T(s' | 0.8, a) U^{\tilde{\pi}}(s') ds'$$

We can now compute $Q^{\tilde{\pi}}(0.8, a)$ for all $a \in A$, beginning with $a = 0$. Note that $\mathcal{U}([s + a, s + a + 1]) = 1 \forall a, s$ by definition of the uniform distribution.

$a = 0$:

$$Q^{\tilde{\pi}}(0.8, 0) = R(0.8, 0) + \int_{-\infty}^{\infty} T(s' | 0.8, 0) U^{\tilde{\pi}}(s') ds'$$

$$Q^{\tilde{\pi}}(0.8, 0) = (-\mathbb{1}(0.8 \leq 1) - 0) + \int_{-\infty}^{\infty} \mathcal{U}([0.8, 1.8]) (-2 \cdot \mathbb{1}(s' \leq 1)) ds'$$

$$Q^{\tilde{\pi}}(0.8, 0) = -1 + \int_{-\infty}^{\infty} (-2 \cdot \mathbb{1}(s' \leq 1)) ds'$$

$$Q^{\tilde{\pi}}(0.8, 0) = -1 + \int_{0.8}^1 -2 ds' = -1 - 2(0.2) = -1.4$$

$a = 0.1$:

$$\begin{aligned}
 Q^{\tilde{\pi}}(0.8, 0.1) &= R(0.8, 0.1) + \int_{-\infty}^{\infty} T(s' | 0.8, 0.1) U^{\tilde{\pi}}(s') ds' \\
 Q^{\tilde{\pi}}(0.8, 0.1) &= (-\mathbb{1}(0.8 \leq 1) - 0.1) + \int_{-\infty}^{\infty} (-2 \cdot \mathbb{1}(s' \leq 1)) ds' \\
 Q^{\tilde{\pi}}(0.8, 0.1) &= -1.1 + \int_{0.9}^{1.9} (-2 \cdot \mathbb{1}(s' \leq 1)) ds' \\
 Q^{\tilde{\pi}}(0.8, 0.1) &= -1.1 + \int 0.9^1 - 2 ds' = -1.1 - 2(0.1) = -1.3
 \end{aligned}$$

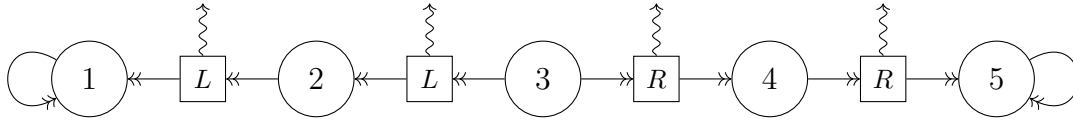
$a = 1$:

$$\begin{aligned}
 Q^{\tilde{\pi}}(0.8, 1) &= R(0.8, 1) + \int_{-\infty}^{\infty} T(s' | 0.8, 1) U^{\tilde{\pi}}(s') ds' \\
 Q^{\tilde{\pi}}(0.8, 1) &= (-\mathbb{1}(0.8 \leq 1) - 1) + \int_{-\infty}^{\infty} (-2 \cdot \mathbb{1}(s' \leq 1)) ds' \\
 Q^{\tilde{\pi}}(0.8, 1) &= -2 + \int_{1.8}^{2.8} (-2 \cdot \mathbb{1}(s' \leq 1)) ds' \\
 Q^{\tilde{\pi}}(0.8, 1) &= -2
 \end{aligned}$$

Thus the action which minimizes $Q^{\tilde{\pi}}(0.8, a)$ is $\boxed{a = 0.1}$.

Problem 2:

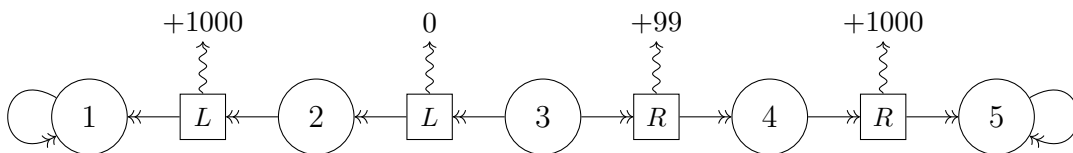
Given the following MDP:



We are asked to disprove the following statement:

If a policy π satisfies $|Q^*(s, \pi^*(s)) - Q^*(s, \pi(s))| \leq \beta$ for all $s \in S$ for some $\beta > 0$, then it immediately follows that $|R(s, \pi^*(s)) - R(s, \pi(s))| \leq \beta$ for any $s \in S$.

In order to find a counter example, we must find a reward function, R a policy π , and a value β that disproves the claim. Since all transitions are deterministic, and every state other than $s = 3$ only has a single action we only have to choose $\pi(3)$ and $\pi^*(3)$, the optimal policy. When choosing $R(s, a)$, we will ensure that one action at $s = 3$ is clearly better than the other action, that way our policy will differ from the optimal policy which is necessary to disprove the statement. To disprove the claim, consider the following reward function for our MDP:



It is clear that the optimal policy at $s = 3$ is $\pi^*(3) = R$, so we will choose our policy to be $\pi(3) = L$. Next, we will use the Bellman Backup algorithm in order to compute values for $Q(s, \pi)$. Since we have two terminal states, $s = 1$ and $s = 5$, we will start there and work back until we can compute $Q(3, \pi) = R(3, \pi(3)) + 0.9U^\pi(s')$ for both the optimal policy and non-optimal policy,

S	A	Q^*	V^*
1	N/A	0	0
5	N/A	0	0
4	R	1000	1000
2	L	1000	1000
3	R	$99 + 0.9(1000) = 999$	999
3	L	$0.9(1000) = 900$	

Thus, we get $Q(3, \pi^*(3)) = 999$ and $Q(3, \pi(3)) = 900$. Finally, let's select $\beta = 99.5$. Our policy satisfies $|Q^*(3, \pi^*(3)) - Q^*(3, \pi(3))| \leq \beta \rightarrow |999 - 900| \leq 99.5 \rightarrow 99 \leq 99.5$. The original claim states that it immediately follows that $|R(3, \pi^*(3)) - R(3, \pi(3))| \leq \beta$. However, we see that $|100 - 0| = 100 \not\leq 99.5$. Thus, we have found a counter example for the claim.

Problem 3:

- a) Using the code provided to us, I was able to achieve a mean discounted reward estimate of -95.63 reward units with 260 Monte Carlo trials. To compute the SEM, I used the following formula,

$$SEM = \frac{\sigma}{\sqrt{n}}$$

where σ is the standard deviation of the reward estimate, and n is the number of Monte Carlo trials. With $n = 260$ MC trials, I achieved an $SEM = 4.98$.

- b) Given the information that goal cells were 20 cells apart, i.e. at $[20, 20]$, $[20, 40]$, $[40, 20]$, etc. I created a heuristic policy which given a state s , first finds the closest goal cell in the 60×60 grid, then chooses the appropriate action in order to move towards that goal cell. Code for my heuristic policy can be seen in the smart_policy function below. Using again $n = 260$ MC trials, I was able to achieve a mean discounted reward estimate of -29.30 and a $SEM = 3.75$.

3.1 Code For Problem 3

```

1 using DMUStudent.HW3: HW3, DenseGridWorld, visualize_tree
2 using POMDPs: actions, @gen, isterminal, discount, statetype, actiontype,
   simulate, states, initialstate
3 using D3Trees: inchrome, inbrowser
4 using StaticArrays: SA
5 using Statistics: mean, std
6 using BenchmarkTools: @btime
7
8 m = HW3.DenseGridWorld(seed = 3)
9
10 function rollout(mdp, policy_function, s0, max_steps=100)
11     r_total = 0.0

```

```

12     t = 0
13     s = s0
14     while !isterminal(mdp, s) && t < max_steps
15         a = policy_function(mdp, s) # replace this with a policy
16         s, r = @gen(:sp,:r)(mdp, s, a)
17         r_total += discount(m)^t*r
18         t += 1
19     end
20     return r_total # replace this with the reward
21 end
22
23 ### Part a)
24 function heuristic_policy(m, s)
25     # put a smarter heuristic policy here
26     return rand(actions(m))
27 end
28
29 function smart_policy(m,s)
30     x,y = s
31
32     AS = [
33         (:down, (x, y-1)),
34         (:up, (x, y+1)),
35         (:right, (x+1, y)),
36         (:left, (x-1, y))
37     ]
38
39
40     # Find closest goal from CURRENT state
41     goals = [[20,20], [20,40], [40,20], [40,40]]
42     closest_goal_cell = goals[argmin([sqrt((x - g[1])^2 + (y - g[2])^2)
43         for g in goals])]
44     #@show s
45     #@show closest_goal_cell
46
47     # Find which next state is closest to that goal
48     distances_to_goal = [sqrt((next_s[1] - closest_goal_cell[1])^2 + (
49         next_s[2] - closest_goal_cell[2])^2) for (a,next_s) in AS]
50
51     #@show distances_to_goal
52     #@show AS[argmin(distances_to_goal)][1]
53     return AS[argmin(distances_to_goal)][1]
54 end
55
56 # This code runs monte carlo simulations: you can calculate the mean and
57 # standard error from the results
58 # Part a)
59 n = 260
60
61 results = [rollout(m, heuristic_policy, rand(initialstate(m))) for _ in 1:
62     n]
63
64 @show mean(results)

```

```

62 @show std(results)/sqrt(n)
63
64 # part b)
65 n = 260
66 results = [rollout(m, smart_policy, rand(initialstate(m))) for _ in 1:n]
67
68 @show mean(results)
69 @show std(results)/sqrt(n)

```

Problem 4:

After 7 iterations of Monte Carlo Tree Search with a depth of 100 and $c = 2$, we get the tree seen in Figure 1. The actions which lead to the highest Q values from the starting state are the up and right actions. This is because these actions move the state closest the goal cell at [20, 20]. Since I used my heuristic policy outlined in problem 3 for the rollout simulation in MCTS, these actions will be selected the most, which we also see in the tree. Note that the value I chose for c was arbitrarily chosen with no tuning. It was tuned in later problems.

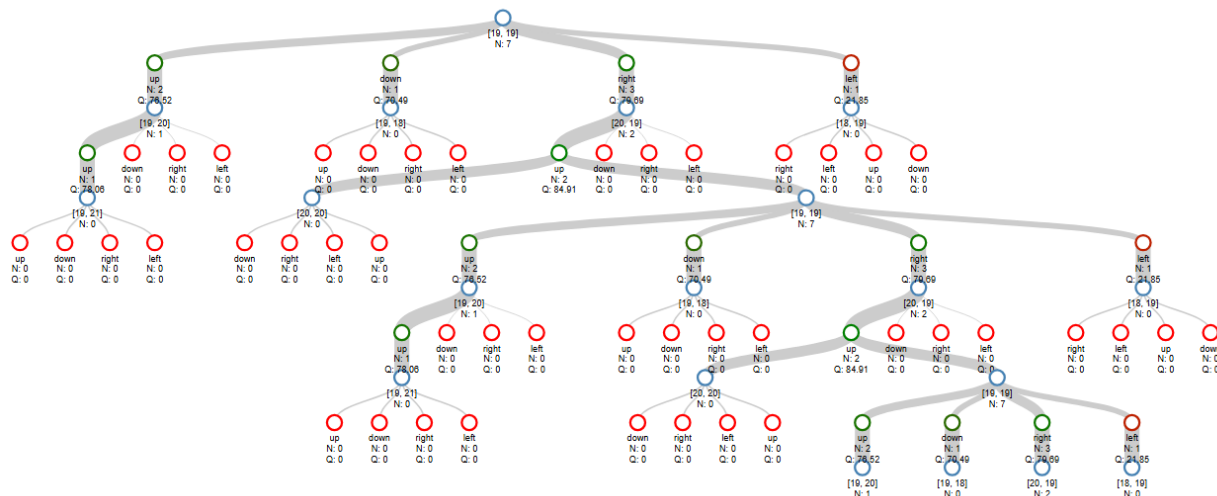


Figure 1: Monte Carlo Tree Search Decision Tree After 7 iterations

4.1 Code For Problem 4

```

1
2 using DMUStudent.HW3: HW3, DenseGridWorld, visualize_tree
3 using POMDPs: actions, @gen, isterminal, discount, statetype, actiontype,
  simulate, states, initialstate
4 using D3Trees: inchrome, inbrowser
5 using StaticArrays: SA
6 using Statistics: mean
7 using BenchmarkTools: @btime
8
9 #####

```

```

10 # Question 4
11 #####
12
13 function rollout(mdp, policy_function, s0, max_steps=100)
14     r_total = 0.0
15     t = 0
16     s = s0
17     while !isterminal(mdp, s) && t < max_steps
18         a = policy_function(mdp, s) # replace this with a policy
19         s, r = @gen(:sp,:r)(mdp, s, a)
20         r_total += discount(m)^t*r
21         t += 1
22     end
23     return r_total # replace this with the reward
24 end
25
26
27
28 function smart_policy(m,s)
29     x,y = s
30
31     AS = [
32         (:down, (x, y-1)),
33         (:up, (x, y+1)),
34         (:right, (x+1, y)),
35         (:left, (x-1, y))
36     ]
37
38
39     # Find closest goal from CURRENT state
40     goals = [[20,20], [20,40], [40,20], [40,40]]
41     closest_goal_cell = goals[argmin([sqrt((x - g[1])^2 + (y - g[2])^2)
42         for g in goals])]
43     #@show s
44     #@show closest_goal_cell
45
46     # Find which next state is closest to that goal
47     distances_to_goal = [sqrt((next_s[1] - closest_goal_cell[1])^2 + (
48         next_s[2] - closest_goal_cell[2])^2) for (a,next_s) in AS]
49
50     #@show distances_to_goal
51     #@show AS[argmin(distances_to_goal)][1]
52     return AS[argmin(distances_to_goal)][1]
53
54 end
55
56 function simulate_MCTS( $\pi$ , s, d = 100)
57
58     # Unpack
59     m =  $\pi$ .m
60
61     if d <= 0 # Base case
62
63         return rollout(m, smart_policy, s) # U(s)

```

```

62     end
63
64     A = actions(m)
65      $\gamma$  = discount(m)
66
67     # If we haven't checked the actions at state s yet, assign all dict
68     # values to 0
69     if !haskey(n, (s, first(A)))
70         for a in A
71              $\pi$ .n[(s,a)] = 0
72              $\pi$ .q[(s,a)] = 0.0
73         end
74         # Value function estimate
75         return rollout(m, smart_policy, s)
76     end
77
78     # Explore decision tree
79     a = explore( $\pi$ , s)
80     sp, r = @gen(:sp, :r)(m, s, a)
81     Q = r +  $\gamma$  * simulate_MCTS( $\pi$ , sp, d-1)
82
83      $\pi$ .n[(s,a)] += 1
84      $\pi$ .q[(s,a)] += (Q -  $\pi$ .q[(s,a)]) /  $\pi$ .n[(s,a)]
85     tval = get( $\pi$ .t, (s, a, sp), 0)
86      $\pi$ .t[(s,a,sp)] = tval + 1
87
88     return Q
89 end
90
91 function explore( $\pi$ , s)
92     m =  $\pi$ .m
93     n =  $\pi$ .n
94     q =  $\pi$ .q
95     c =  $\pi$ .c
96     A = actions(m)
97
98     bonus = (Nsa, Ns) -> Nsa == 0 ? Inf : sqrt(log(Ns)/Nsa)
99     Ns = sum(n[(s,a)] for a in A)
100     return argmax(a -> q[(s,a)] + c*bonus(n[(s,a)], Ns), A)
101 end
102
103
104 m = DenseGridWorld()
105
106 S = statetype(m)
107 A = actiontype(m)
108
109 # These would be appropriate containers for your Q, N, and t dictionaries:
110 n = Dict{Tuple{S, A}, Int}()
111 q = Dict{Tuple{S, A}, Float64}()
112 t = Dict{Tuple{S, A, S}, Int}()
113 c = 2
114

```

```

115 mutable struct MCTS
116     m
117     n
118     q
119     t
120     c
121 end
122
123  $\pi$  = MCTS(m,n,q,t,c)
124
125 # This is an example state - it is a StaticArrays.SVector{2, Int}
126 s = SA[19,19]
127
128 @assert s isa statetype(m)
129
130 for _ in 1:7
131     simulate_MCTS( $\pi$ , s)
132 end
133
134 inchrome(visualize_tree( $\pi$ .q,  $\pi$ .n,  $\pi$ .t, s))

```

Problem 5:

5.1 Code For Problem 5

Using my MCTS code from problem 4 I developed two new functions, `policy_`, and `eval_online_MCTS`. The function `policy_` selects an actions based on the Q estimate provided by 1000 iterations of MCTS with a depth of 20. The function `eval_online_MCTS` runs the 100 100-step MC trials of our functions, terminating when a terminal state is reached. This resulted in a mean discounted reward of -17.02 and $SEM = 7.77$

```

1 using DMUStudent.HW3: HW3, DenseGridWorld, visualize_tree
2 using POMDPs: actions, @gen, isterminal, discount, statetype, actiontype,
   simulate, states, initialstate
3 using D3Trees: inchrome, inbrowser
4 using StaticArrays: SA
5 using Statistics: mean, std
6 using BenchmarkTools: @btime
7
8 #####
9 # Question 5
10 #####
11
12
13 ### rollout sim from state s0
14 function rollout( $\pi$ , policy_function, s0, max_steps=20)
15     r_total = 0.0
16     t = 0
17     s = s0
18     while !isterminal( $\pi$ .m, s) && t < max_steps
19         a = policy_function( $\pi$ , s) # replace this with a policy
20         s, r = @gen(:sp,:r)( $\pi$ .m, s, a)
21         r_total += discount( $\pi$ .m)^t*r

```

```

22     t += 1
23     end
24     return r_total # replace this with the reward
25 end
26
27
28 ### Policy for DenseGridWorld
29 function smart_policy(m,s)
30     x,y = s
31
32     AS = [
33         (:down, (x, y-1)),
34         (:up, (x, y+1)),
35         (:right, (x+1, y)),
36         (:left, (x-1, y))
37     ]
38
39
40     # Find closest goal from CURRENT state
41     goals = [[20,20], [20,40], [40,20], [40,40]]
42     closest_goal_cell = goals[argmin([(x - g[1])^2 + (y - g[2])^2 for g in
43         goals])]
44     #@show s
45     #@show closest_goal_cell
46
47     # Find which next state is closest to that goal
48     distances_to_goal = [(next_s[1] - closest_goal_cell[1])^2 + (next_s[2]
49         - closest_goal_cell[2])^2 for (a,next_s) in AS]
50
51     #@show distances_to_goal
52     #@show AS[argmin(distances_to_goal)][1]
53     return AS[argmin(distances_to_goal)][1]
54 end
55
56 ### main MCTS simulation
57 function simulate_MCTS( $\pi$ , s, d = 100)
58
59     # Unpack
60     m =  $\pi$ .m
61
62     if d <= 0 # Base case
63         return rollout( $\pi$ , smart_policy, s) # U(s)
64     end
65
66     A = actions(m)
67      $\gamma$  = discount(m)
68
69     # If we haven't checked the actions at state s yet, assign all dict
70     # values to 0
71     if !haskey( $\pi$ .n, (s, first(A)))
72         for a in A
73              $\pi$ .n[(s,a)] = 0
74              $\pi$ .q[(s,a)] = 0.0
75         end
76     end
77
78     # Rollout
79     s = first(A)
80     r = 0
81     while true
82         (a,next_s) = smart_policy(m,s)
83         r = r +  $\gamma$  * (U(next_s) - U(s))
84         s = next_s
85     end
86
87     # Update
88      $\pi$ .n[(s,a)] = r
89      $\pi$ .q[(s,a)] =  $\gamma$  * ( $\pi$ .q[(s,a)] + (1 -  $\gamma$ ) * r)
90
91     return  $\pi$ .q[(s,a)]
92 end

```

```

73
74     end
75     # Value function estimate
76     return rollout( $\pi$ , smart_policy, s)
77 end
78
79 # Explore decision tree
80 a = explore( $\pi$ , s)
81 sp, r = @gen(:sp, :r)(m, s, a)
82 Q = r +  $\gamma$  * simulate_MCTS( $\pi$ , sp, d-1)
83
84  $\pi$ .n[(s,a)] += 1
85  $\pi$ .q[(s,a)] += (Q -  $\pi$ .q[(s,a)]) /  $\pi$ .n[(s,a)]
86 tval = get( $\pi$ .t, (s, a, sp), 0)
87  $\pi$ .t[(s,a,sp)] = tval + 1
88
89 return Q
90 end
91
92 ### Explore decision tree for mcts
93 function explore( $\pi$ , s)
94     m =  $\pi$ .m
95     n =  $\pi$ .n
96     q =  $\pi$ .q
97     c =  $\pi$ .c
98     A = actions(m)
99
100     bonus = (Nsa, Ns) -> Nsa == 0 ? Inf : sqrt(log(Ns)/Nsa)
101     Ns = sum(n[(s,a)] for a in A)
102     return argmax(a -> q[(s,a)] + c*bonus(n[(s,a)]), Ns), A)
103
104 end
105
106 ### Get an action based on MCTS tree
107 function policy_( $\pi$ , s)
108     for k in 1:1000
109         simulate_MCTS( $\pi$ , s) # 1000 iterations to choose each action, d =
110                               100 by default
111     end
112     return argmax(a ->  $\pi$ .q[(s,a)], actions( $\pi$ .m))
113 end
114
115 function eval_online_MCTS(m)
116     all_rewards = []
117
118     for i in 1:100 # 100 MC simulations
119
120         # Declare/Redeclare necessary params
121         n = Dict{Tuple{S, A}, Int}()
122         q = Dict{Tuple{S, A}, Float64}()
123         t = Dict{Tuple{S, A, S}, Int}()
124         c = sqrt(2)
125
126          $\pi$  = MCTS(m, n, q, t, c)

```

```

126
127     s = rand(initialstate(m))
128     total_reward = 0.0
129
130     for j in 1:100 # 100 step unless it reaches a terminal state
131         a = policy_( $\pi$ ,s) # Calls MCTS to plan next actions
132
133         sp, r = @gen(:sp,:r)( $\pi$ .m, s, a) # Take the next step
134
135         total_reward += r
136
137         s = sp
138
139         if isterminal(m,s) break end# Break out, run next trial
140
141     end
142
143     push!(all_rewards, total_reward)
144
145 end
146
147 return all_rewards
148
149 end
150
151 m = DenseGridWorld()
152
153 S = statetype(m)
154 A = actiontype(m)
155
156 # These would be appropriate containers for your Q, N, and t dictionaries:
157 n = Dict{Tuple{S, A}, Int}{}
158 q = Dict{Tuple{S, A}, Float64}{}
159 t = Dict{Tuple{S, A, S}, Int}{}
160 c = sqrt(2)
161
162 mutable struct MCTS
163     m
164     n
165     q
166     t
167     c
168 end
169
170 results = eval_online_MCTS(m)
171
172 @show mean(results)
173 @show std(results)/sqrt(100)

```

Problem 6:

For the challenge problem, I mainly focused on tuning the value of c so that my code would reach a score of 50. I opted to stick with the MCTS code developed in problem 4, as well as the rollout

policy I implemented in problem 3. I modified the rollout policy to account for the added goal cells that came with the larger grid world. While this rollout policy may be slow, as it includes loops over the goal cells and actions, I wanted a policy that was likely close to the optimal policy, and it likely did not cause much of a slow down in compute. As mentioned earlier, I mainly focused on tuning c for my MCTS code to meet the requirements. I began with the value I initially started with, $c = \sqrt{2}$ and increased the order by 10 until it achieved a score of 50. I chose to increase the value of c instead of decreasing it because I believed that encouraging the tree to explore other actions would result in the tree search trying the worse actions more often, get a lower value for Q , and ultimately end up choosing the best action in the function policy. I kept iterating this process until I recieved a score of 50, which I achieved with a value of $c = 100\sqrt{2}$. I chose to not vary depth as I found c to have a much larger impact on the tree's performance, thus I kept it at a value of 15.

6.1 Code For Problem 6

```

1 using DMUStudent.HW3: HW3, DenseGridWorld, visualize_tree
2 using POMDPs: actions, @gen, isterminal, discount, statetype, actiontype,
   simulate, states, initialstate
3 using D3Trees: inchrome, inbrowser
4 using StaticArrays: SA
5 using Statistics: mean, std
6 using BenchmarkTools: @btime
7
8 #####
9 # Question 6
10 #####
11
12
13 ### rollout sim from state s0
14 function rollout( $\pi$ , policy_function, s0, max_steps=100)
15     r_total = 0.0
16     t = 0
17     s = s0
18     while !isterminal( $\pi$ .m, s) && t < max_steps
19         a = policy_function( $\pi$ , s) # replace this with a policy
20         s, r = @gen(:sp,:r)( $\pi$ .m, s, a)
21         r_total += discount( $\pi$ .m)^t*r
22         t += 1
23     end
24     return r_total # replace this with the reward
25 end
26
27
28 ### Policy for DenseGridWorld
29 function smart_policy(m,s)
30     x,y = s
31
32     AS = [
33         (:down, (x, y-1)),
34         (:up, (x, y+1)),
35         (:right, (x+1, y)),
36         (:left, (x-1, y))

```

```

37 ]
38
39
40 # Find closest goal from CURRENT state
41 goals = [[20,20], [20,40], [20,60], [20,80], [40,20], [40,40],
42           [40,60], [40,80], [60,20], [60,40], [60,60], [60,80], [80,20],
43           [80,40], [80,60], [80,80]]
44 closest_goal_cell = goals[argmin([(x - g[1])**2 + (y - g[2])**2 for g in
45                                   goals])]
46 # @show s
47 # @show closest_goal_cell
48
49 # Find which next state is closest to that goal
50 distances_to_goal = [(next_s[1] - closest_goal_cell[1])**2 + (next_s[2]
51                       - closest_goal_cell[2])**2 for (a, next_s) in AS]
52
53 # @show distances_to_goal
54 # @show AS[argmin(distances_to_goal)][1]
55 return AS[argmin(distances_to_goal)][1]
56
57 end
58
59 ### main MCTS simulation
60 function simulate_MCTS( $\pi$ , s, d = 15)
61
62     # Unpack
63     m =  $\pi$ .m
64
65     if d <= 0 # Base case
66         return rollout( $\pi$ , smart_policy, s) # U(s)
67     end
68
69     A = actions(m)
70      $\gamma$  = discount(m)
71
72     # If we haven't checked the actions at state s yet, assign all dict
73     # values to 0
74     if !haskey( $\pi$ .n, (s, first(A)))
75         for a in A
76              $\pi$ .n[(s,a)] = 0
77              $\pi$ .q[(s,a)] = 0.0
78         end
79
80         # Value function estimate
81         return rollout( $\pi$ , smart_policy, s)
82     end
83
84     # Explore decision tree
85     a = explore( $\pi$ , s)
86     sp, r = @gen(:sp, :r)(m, s, a)
87     Q = r +  $\gamma$  * simulate_MCTS( $\pi$ , sp, d-1)
88
89      $\pi$ .n[(s,a)] += 1
90      $\pi$ .q[(s,a)] += (Q -  $\pi$ .q[(s,a)]) /  $\pi$ .n[(s,a)]

```

```

86     tval = get( $\pi$ .t, (s, a, sp), 0)
87      $\pi$ .t[(s,a,sp)] = tval + 1
88
89     return Q
90 end
91
92 ### Explore decision tree for mcts
93 function explore( $\pi$ , s)
94     m =  $\pi$ .m
95     n =  $\pi$ .n
96     q =  $\pi$ .q
97     c =  $\pi$ .c
98     A = actions(m)
99
100     bonus = (Nsa, Ns) -> Nsa == 0 ? Inf : sqrt(log(Ns)/Nsa)
101     Ns = sum(n[(s,a)] for a in A)
102     return argmax(a -> q[(s,a)] + c*bonus(n[(s,a)], Ns), A)
103
104 end
105
106 ### Get an action based on MCTS tree
107 function select_action(m, s)
108     S = statetype(m)
109     A = actiontype(m)
110
111     # These would be appropriate containers for your Q, N, and t
112     # dictionaries:
113     n = Dict{Tuple{S, A}, Int}{}
114     q = Dict{Tuple{S, A}, Float64}{}
115     t = Dict{Tuple{S, A, S}, Int}{}
116     c = sqrt(2)*100
117
118      $\pi$  = MCTS(m,n,q,t,c)
119
120     for k in 1:1000
121         simulate_MCTS( $\pi$ ,s) # 1000 iterations to choose each action, d =
122                               100 by default
123     end
124     return argmax(a ->  $\pi$ .q[(s,a)], actions( $\pi$ .m))
125 end
126
127 m = DenseGridWorld()
128
129 mutable struct MCTS
130     m
131     n
132     q
133     t
134     c
135 end
136
137 s0 = rand(initialstate(m))
138 a = select_action(m, s0)
139

```

```
138 @show s0
139 @show a
140
141 HW3.evaluate(select_action,"bijan.jourabchi@colorado.edu")
```